

GRAPH ALGORITHMS

MASTERCLASS

FAANG Discovery Journey — First Principles

🏠 THE REAL-WORLD STORY

Imagine Google Maps connecting cities by highway roads. Bangalore is connected to Mysore, Mysore to Goa. How do we model 1,000 cities?

💡 **THE GRAPH DISCOVERY AHA**

- 1. Single City -> **Node**
- 2. Highway Road -> **Edge**
- 3. Connected Cities -> **Neighbours**
- 4. Collection of Nodes & Edges -> **A Graph is Born!**

😞 WHY EXISTING STRUCTURES FAIL

- **Arrays?** Cannot express complex 2D networks.
- **Trees?** Trees enforce 1 parent and no cycles. Graphs allow multiple parents and cycles!
- **Aha Moment:** A Tree is just a restricted Graph without cycles! A 2D Grid is just a Graph where each cell has 4 directional neighbours!

➡ UNIVERSAL GRAPH THINKING

- The 5 Questions for EVERY Graph Problem:**
1. What are my **Nodes**?
 2. What are my **Edges**?
 3. Directed or Undirected? Weighted or Unweighted?
 4. Do I need a **Visited Set** to prevent infinite cycle loops?
 5. Is this **DFS** (all paths) or **BFS** (shortest path)?

🔧 GRAPH REPRESENTATIONS (BUILD IT YOURSELF)

Representation	Space	Edge Check `(u, v)`	Neighbour Iteration
Edge List `[ [u, v] ]`	$O(E)$	$O(E)$ linear search	$O(E)$ scan
Adjacency Matrix `int[V][V]`	$O(V^2)$	$O(1)$ instant lookup	$O(V)$ row scan
Adjacency List `List<`	$O(V + E)$	$O(\text{deg}(u))$	$O(\text{deg}(u))$ optimal!

⚡ COURSE SCHEDULE INDEGREE CONVERSION

In *Course Schedule (LC 207)*, prerequisites arrive as `[course, prereq]`. To take `course`, you must complete `prereq` first!

```
Directional Edge: `prereq -> course`
`adj.get(prereq).add(course);`
`indegree[course]++;` // Increment indegree of dependent course!
```

GRAPH ALGORITHMS MASTERCLASS

Java Master Toolkit & Utility Classes



PATTERN 1 NUMBER OF ISLANDS (2D Grid Directional DFS) e.g. Number of Islands (LC 200)

GRID AS A GRAPH

Grid cell `(r, c)` is a Node. 4-directional moves
`(-1,0), (1,0), (0,-1), (0,1)` are Edges. Sink visited
land `grid[r][c] = '0'`!

TEMPLATE — LC 200

```
public int numIslands(char[][] grid) {
    int count = 0;
    for (int r = 0; r < grid.length; r++) {
        for (int c = 0; c < grid[0].length; c++) {
            if (grid[r][c] == '1') {
                count++;
                dfs(grid, r, c); // Sink connected component!
            }
        }
    }
    return count;
}

private void dfs(char[][] g, int r, int c) {
    if (r < 0 || r >= g.length || c < 0 || c >= g[0].length || g[r][c] == '0') return;
    g[r][c] = '0'; // Sink land!
    dfs(g, r + 1, c); dfs(g, r - 1, c);
    dfs(g, r, c + 1); dfs(g, r, c - 1);
}
```

PATTERN 2

CLONE GRAPH (HashMap Node Mapping + DFS) e.g. Clone Graph (LC 133)

MAP ORIGINAL -> CLONE

Map `oldNode -> newNode`. If node already cloned, return from map. Otherwise create clone node and recursively clone all neighbours!

TEMPLATE — LC 133

```
private Map<Node, Node> visited = new HashMap<>();
public Node cloneGraph(Node node) {
    if (node == null) return null;
    if (visited.containsKey(node)) return visited.get(node);
    Node clone = new Node(node.val);
    visited.put(node, clone);
    for (Node neighbor : node.neighbors) {
        clone.neighbors.add(cloneGraph(neighbor));
    }
    return clone;
}
```

PATTERN 4 DIJKSTRA'S SHORTEST PATH (Min-Heap Weighted Search) e.g. Network Delay Time (LC 743)

MIN-HEAP DISTANCE RELAXATION

PriorityQueue stores `{dist, node}`. Always expand minimum distance node. Relax edge `dist[v] = dist[u] + weight` if shorter! Runs in $O((V + E) \log V)$.

TEMPLATE — LC 743

```
public int networkDelayTime(int[][] times, int n, int k) {
    List<List<int[]>> adj = new ArrayList<>();
    for (int i = 0; i <= n; i++) adj.add(new ArrayList<>());
    for (int[] t : times) adj.get(t[0]).add(new int[]{t[1], t[2]});
    int[] dist = new int[n + 1];
    Arrays.fill(dist, Integer.MAX_VALUE);
    dist[k] = 0;
    PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[0] - b[0]);
    pq.offer(new int[]{0, k});
    while (!pq.isEmpty()) {
        int[] cur = pq.poll();
        int d = cur[0], u = cur[1];
        if (d > dist[u]) continue;
        for (int[] edge : adj.get(u)) {
            int v = edge[0], weight = edge[1];
            if (dist[u] + weight < dist[v]) {
                dist[v] = dist[u] + weight;
                pq.offer(new int[]{dist[v], v});
            }
        }
    }
    int maxDist = 0;
    for (int i = 1; i <= n; i++) {
        if (dist[i] == Integer.MAX_VALUE) return -1;
        maxDist = Math.max(maxDist, dist[i]);
    }
    return maxDist;
}
```

PATTERN 5

UNION-FIND DSU (Path Compression & Rank) e.g. Redundant Connection (LC 684)

DYNAMIC COMPONENT MERGE

`parent[i] = find(parent[i])` flattens trees to height 1. `union(u, v)` merges components. If `find(u) == find(v)`, cycle detected in $O(\alpha(N))$ time!

TEMPLATE — LC 684

```
class DSU {
    int[] parent, rank;
    public DSU(int n) {
        parent = new int[n]; rank = new int[n];
        for (int i = 0; i < n; i++) parent[i] = i;
    }
    public int find(int i) {
        if (parent[i] == i) return i;
        return parent[i] = find(parent[i]);
    }
    public boolean union(int i, int j) {
        int rI = find(i), rJ = find(j);
        if (rI == rJ) return false; // Cycle detected!
        if (rank[rI] < rank[rJ]) parent[rI] = rJ;
        else if (rank[rI] > rank[rJ]) parent[rJ] = rI;
        else { parent[rJ] = rI; rank[rI]++; }
        return true;
    }
}
```

GRAPH ALGORITHMS

MASTERCLASS

Decision Tree & Trigger Words

COMPLETE GRAPH ALGORITHMS PROBLEM LADDER SUMMARY					
LC #	Problem	Pattern	Time	Space	Diff
733	Flood Fill	2D Grid DFS Recurrence	$O(M \cdot N)$	$O(M \cdot N)$	E
200	Number of Islands	2D Grid DFS Component Sinking	$O(M \cdot N)$	$O(M \cdot N)$	M
133	Clone Graph	HashMap Node Copy + DFS	$O(V + E)$	$O(V)$	M
207	Course Schedule	Kahn's BFS Topological Sort	$O(V + E)$	$O(V + E)$	M
994	Rotting Oranges	Multi-Source BFS Queue	$O(M \cdot N)$	$O(M \cdot N)$	M
743	Network Delay Time	Dijkstra's Min-Heap Shortest Path	$O((V + E) \log V)$	$O(V + E)$	M
684	Redundant Connection	Union-Find (DSU) Cycle Check	$O(N \cdot \alpha(N))$	$O(N)$	M

GRAPH ALGORITHMS

MASTERCLASS

Curated Problem Ladder — Part 1

PAGE 8 OF 10

PROBLEM LADDER · EASY &
MEDIUM

LC 733 · Flood Fill

EASY

Pattern: 2D Grid DFS
Key Insight: Recurse 4 directions if color matches.

```
public int[][] floodFill(int[][] img, int sr, int sc, int newColor) {
    int orig = img[sr][sc];
    if (orig != newColor) dfs(img, sr, sc, orig, newColor);
    return img;
}
private void dfs(int[][] img, int r, int c, int orig, int newC) {
    if (r < 0 || r >= img.length || c < 0 || c >= img[0].length || img[r][c] != orig) return;
    img[r][c] = newC;
    dfs(img, r+1, c, orig, newC); dfs(img, r-1, c, orig, newC);
    dfs(img, r, c+1, orig, newC); dfs(img, r, c-1, orig, newC);
}
```

LC 200 · Number of Islands

MEDIUM

Pattern: Grid DFS Sinking
Key Insight: Sink land `grid[r][c] = '0'`.

```
public int numIslands(char[][] g) {
    int count = 0;
    for (int r = 0; r < g.length; r++)
        for (int c = 0; c < g[0].length; c++)
            if (g[r][c] == '1') { count++; dfs(g, r, c); }
    return count;
}
private void dfs(char[][] g, int r, int c) {
    if (r < 0 || r >= g.length || c < 0 || c >= g[0].length || g[r][c] == '0') return;
    g[r][c] = '0';
    dfs(g, r+1, c); dfs(g, r-1, c);
    dfs(g, r, c+1); dfs(g, r, c-1);
}
```

LC 133 · Clone Graph

MEDIUM

Pattern: Map Node Copy
Key Insight: Map `oldNode` -> `newNode`.

```
private Map<Node, Node> vis = new HashMap<>();
public Node cloneGraph(Node node) {
    if (node == null) return null;
    if (vis.containsKey(node)) return vis.get(node);
    Node clone = new Node(node.val);
    vis.put(node, clone);
    for (Node n : node.neighbors) clone.neighbors.add(cloneGraph(n));
    return clone;
}
```


GRAPH ALGORITHMS

MASTERCLASS

Curated Problem Ladder — Part 2

PAGE 9 OF 10

PROBLEM LADDER · MEDIUM &
HARD

LC 207 · Course Schedule

MEDIUM

Pattern: Kahn's BFS
Topo

Key Insight: `prereq -> course`, queue 0 indegree.

```
public boolean canFinish(int n, int
[] pre) {
    List<List<Integer>> adj = new Arr
ayList<>(); int[] deg = new int[n];
    for (int i = 0; i < n; i++) adj.a
dd(new ArrayList<>());
    for (int[] p : pre) { adj.get(p
[1]).add(p[0]); deg[p[0]]++; }
    Queue<Integer> q = new ArrayDeque
<>();
    for (int i = 0; i < n; i++) if (d
eg[i] == 0) q.offer(i);
    int c = 0;
    while (!q.isEmpty()) {
        int u = q.poll(); c++;
        for (int v : adj.get(u)) if (--
deg[v] == 0) q.offer(v);
    }
    return c == n;
}
```

LC 994 · Rotting Oranges

MEDIUM

Pattern: Multi-Source
BFS

Key Insight: Offer all
rotten oranges to
queue at t=0.

```
public int orangesRotting(int[][]
g) {
    Queue<int[]> q = new ArrayDeque<
>(); int fresh = 0, mins = 0;
    for (int r = 0; r < g.length; r+
+)
        for (int c = 0; c < g[0].lengt
h; c++)
            if (g[r][c] == 2) q.offer(new
int[]{r, c});
            else if (g[r][c] == 1) fresh+
+;
    int[][] dirs = {{-1,0},{1,0},{0,-
1},{0,1}};
    while (!q.isEmpty() && fresh > 0)
    {
        int sz = q.size();
        for (int i = 0; i < sz; i++) {
            int[] cur = q.poll();
            for (int[] d : dirs) {
                int nr = cur[0]+d[0], nc =
cur[1]+d[1];
                if (nr>=0 && nr<g.length && nc
```

LC 743 · Network Delay Time

MEDIUM

Pattern: Dijkstra Min-
Heap

Key Insight: Always
expand smallest
distance node.

```
public int networkDelayTime(int[][] t
imes, int n, int k) {
    List<List<int[]>> adj = new ArrayLi
st<>();
    for (int i = 0; i <= n; i++) adj.ad
d(new ArrayList<>());
    for (int[] t : times) adj.get(t
[0]).add(new int[]{t[1], t[2]});
    int[] dist = new int[n + 1]; Array
s.fill(dist, Integer.MAX_VALUE); dist
[k] = 0;
    PriorityQueue<int[]> pq = new Prior
ityQueue<>((a, b) -> a[0] - b[0]);
    pq.offer(new int[]{0, k});
    while (!pq.isEmpty()) {
        int[] cur = pq.poll(); int d = cu
r[0], u = cur[1];
        if (d > dist[u]) continue;
        for (int[] e : adj.get(u)) {
            if (dist[u] + e[1] < dist[e
[0]]) { dist[e[0]] = dist[u] + e[1];
            pq.offer(new int[]{dist[e[0]], e
[0]}); }
        }
    }
    int maxD = 0; for (int i = 1; i <=
n; i++) { if (dist[i] == Integer.MAX_
VALUE) return -1; maxD = Math.max(max
D, dist[i]); }
    return maxD;
}
```

LC 684 · Redundant Connection

MEDIUM

Pattern: DSU Cycle
Check

Key Insight: Edge
creating cycle returns
false on union.

```
public int[] findRedundantConnection
(int[][] edges) {
    DSU dsu = new DSU(edges.length +
1);
    for (int[] e : edges) if (!dsu.unio
n(e[0], e[1])) return e;
    return new int[0];
}
```

GRAPH ALGORITHMS

MASTERCLASS

Dry Run, Proofs & Mastery Cheat Sheet

🔍 DRY RUN — Kahn's Topological Sort (Course Schedule)

Step	q Poll	Neighbours Processed	indegree Array State	Action / Result
1	0 (prereq)	Course 1 (<code>1 <= 0</code>)	<code>indegree[1] = 0</code>	<code>q.offer(1)</code> , <code>count = 1</code>
2	1 (course)	None	<code>indegree = [0, 0]</code>	<code>count = 2</code> ✅ <code>count == numCourses</code>

📐 MATHEMATICAL PROOF — DIJKSTRA'S GREEDY MIN-HEAP PROOF

Claim: When node u is popped from Dijkstra's PriorityQueue, its shortest distance `dist[u]` is finalized.

Proof: Since edge weights are strictly non-negative ($w \geq 0$), any alternative path to u passing through another unvisited node x must have length $\geq \text{dist}[x] \geq \text{dist}[u]$. Thus, no shorter path can be discovered later -> **Exact Proof of Correctness!**

✅ MASTERY CHECKLIST — CAN YOU DO THIS?

- ☐ Explain Graph components from first principles (Cities story)
- ☐ Convert prerequisites array to directed graph & indegrees
- ☐ Write Number of Islands 2D grid DFS in 3 minutes
- ☐ Code Kahn's Topological Sort algorithm using queue
- ☐ Implement Dijkstra's algorithm using PriorityQueue
- ☐ Write Multi-Source BFS for Rotting Oranges
- ☐ Detect graph cycles using Union-Find DSU
- ☐ Prove why Dijkstra fails on negative edge weights

📦 GOLDEN RULES — NEVER FORGET

Rule 1 — Mark Visited at Queue Offer Time

In BFS, ALWAYS mark nodes visited at the exact moment you push them into the queue (`visited[v] = true; q.offer(v)`). Do NOT wait until `q.poll()` or duplicate nodes will pollute your queue!

Rule 2 — Convert `[prereq, course]` Carefully

In Course Schedule problems, double check edge orientation: `[u, v]` means taking `v` requires `u`. Edge must go `u -> v` (`adj.get(u).add(v)` and `indegree[v]++`)!